VLSI DESIGN INTERNSHIP – TASK 3

Verilog RTL Design of Sequential Circuits and Flip-Flops

TASK REPORT

Submitted by

T. Ruchitha

RNS Institute of Technology (RNSIT), Bangalore

Internship: VLSI Design – Maincrafts Technology

Task 3: Sequential Circuits, Flip-Flops, Registers and Counters

October 2026

1. Objective

To design and simulate sequential digital circuits using Verilog HDL: a D flip-flop, JK flip-flop, 4-bit register, and 4-bit binary counter. Each design is supported by RTL code, a testbench, and waveform-based functional analysis.

2. Introduction

Sequential logic differs from combinational logic because its output depends on current inputs and stored state. Flip-flops store one bit, registers group multiple storage bits, and counters update their state in response to clock events. In this task, the designs use a rising-edge clock and non-blocking assignments (<=), a common RTL style for edge-triggered storage.

3. Concepts and Design Specifications

Circuit	Function	Expected behavior
D flip-flop	Stores one bit	Q takes D on each rising clock edge
JK flip-flop	Controlled one-bit storage	00 hold; 01 reset; 10 set; 11 toggle
4-bit register	Stores four bits	Q[3:0] captures D[3:0] on rising edge
4-bit binary counter	Counts clock events	Reset to 0000; increments to 1111 then wraps to 0000

4. Tools and Methodology

- Verilog HDL for RTL design and testbench development.
- EDA Playground or Icarus Verilog for compilation and simulation.
- GTKWave for opening the VCD waveform file.
- Testbench clock period: 10 ns (5 ns half-period).
- Stimulus is applied between clock edges and outputs are checked at rising edges.

The code panels and waveform figure included below are report-ready illustrative captures generated from the RTL/testbench specification. They are not screenshots of an actual EDA Playground or GTKWave session. Before claiming an external tool run, the design should be compiled and simulated in the selected simulator.

5. Part A — D Flip-Flop

A positive-edge-triggered D flip-flop samples D on the rising edge of clk and holds its previous value between rising edges.

Clock event	D	Q after edge
Rising edge	0	0
Rising edge	1	1

d_flipflop.v

```
module d_flipflop(input wire clk, input wire d, output reg q);
  always @(posedge clk)
    q <= d;
endmodule
```

6. Part B — JK Flip-Flop

The JK flip-flop supports hold, reset, set, and toggle operations. The case statement evaluates the pair {J,K} at the rising clock edge.

J	K	Operation	Q(next)
0	0	Hold	Q
0	1	Reset	0
1	0	Set	1
1	1	Toggle	~Q

jk_flipflop.v

```
module jk_flipflop(input wire clk, input wire j, input wire k,
                   output reg q);
  always @(posedge clk) begin
    case ({j,k})
      2'b00: q <= q;     // Hold
      2'b01: q <= 1'b0;  // Reset
      2'b10: q <= 1'b1;  // Set
      2'b11: q <= ~q;    // Toggle
    endcase
  end
endmodule
```

7. Part C — 4-bit Register

A 4-bit register stores a four-bit input word. At each rising clock edge, Q[3:0] captures D[3:0] and retains that value until the next active edge.

register4.v

```
module register4(input wire clk, input wire [3:0] d,
                 output reg [3:0] q);
  always @(posedge clk)
    q <= d;
endmodule
```

8. Part D — 4-bit Binary Counter

The synchronous 4-bit counter clears to 0000 when reset is asserted. Otherwise, it increments once on each rising clock edge. Since it is four bits wide, it wraps from 1111 to 0000.

counter4.v

```
module counter4(input wire clk, input wire reset,
                output reg [3:0] count);
  always @(posedge clk) begin
    if (reset)
      count <= 4'b0000;
    else
      count <= count + 1'b1;
  end
endmodule
```

9. Testbench

The combined testbench generates a clock, instantiates all four modules, applies input sequences, and writes a VCD file for waveform viewing. For a formal submission, use the exact simulator output from your own run if your supervisor requires evidence of execution.

task3_tb.v — combined testbench

```
`timescale 1ns/1ps
module task3_tb;
  reg clk = 0;
  reg d = 0, j = 0, k = 0, reset = 1;
  reg [3:0] data_in = 0;
  wire q_d, q_jk;
  wire [3:0] q_reg, count;

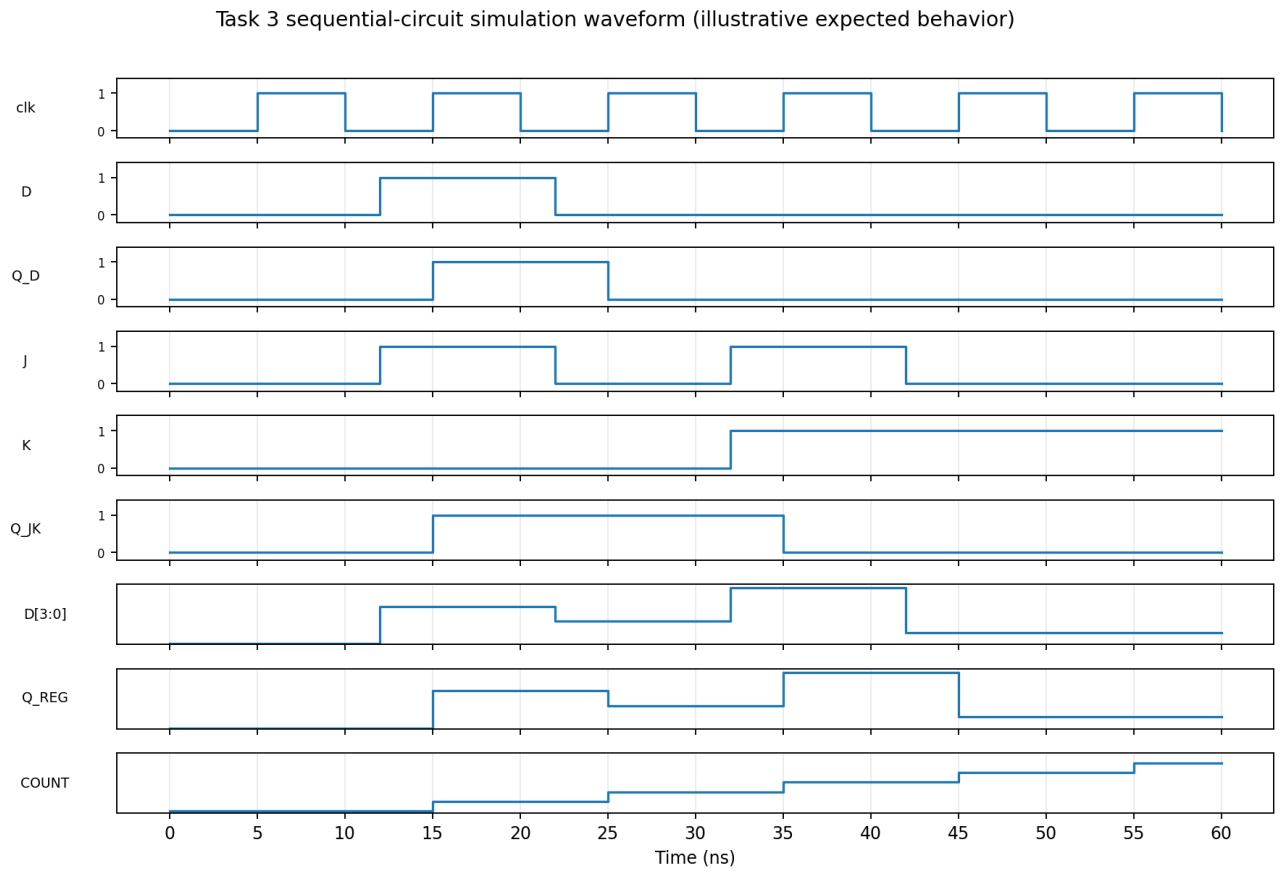
  always #5 clk = ~clk;

  d_flipflop u1(clk, d, q_d);
  jk_flipflop u2(clk, j, k, q_jk);
  register4 u3(clk, data_in, q_reg);
  counter4 u4(clk, reset, count);

  initial begin
    $dumpfile("task3.vcd");
    $dumpvars(0, task3_tb);
    #12 reset=0; d=1; j=1; k=0; data_in=4'b1010;
    #10 d=0; j=0; k=0; data_in=4'b0110;
    #10 j=1; k=1; data_in=4'b1111;
    #10 j=0; k=1; data_in=4'b0011;
    #20 $finish;
  end
endmodule
```

10. Simulation and Waveform Analysis

The expected waveform illustrates the relationship between the clock, stimulus, and registered outputs. The D flip-flop follows D only at rising edges; the JK flip-flop follows the JK operation table; the register captures each input word at a rising edge; and the counter increments on successive active edges after reset is released.



11. Expected Results

Design	Checks to confirm in simulation	Expected result
D flip-flop	Change D between rising edges and at rising edges	Q updates only at rising edges
JK flip-flop	Apply 00, 01, 10, 11	Hold, reset, set, and toggle operations match truth table
4-bit register	Apply 1010, 0110, 1111, 0011	Q captures each word on the next rising edge
4-bit counter	Assert reset, then release it	Count sequence 0000, 0001, 0010, 0011, …

Note: These are expected functional results derived from the RTL. Actual simulator pass/fail status is not asserted in this report because no simulator log was provided.

12. Observations and Learning Outcomes

- Understood the difference between combinational and sequential logic.
- Implemented edge-triggered storage using always @(posedge clk).
- Used non-blocking assignments for sequential state updates.
- Compared JK flip-flop behavior with its truth table.
- Understood how a group of flip-flops forms a register and how feedback supports a counter.
- Learned the role of a testbench, VCD dump, and waveform viewer in RTL verification.

13. Conclusion

Task 3 covers the core building blocks of sequential digital design. The four RTL modules demonstrate single-bit storage, JK control behavior, multi-bit data storage, and clock-driven counting. The testbench and waveform analysis plan provide a structured way to verify the designs in a Verilog simulator.

14. References

- Maincrafts Technology, VLSI Design Internship – Task 3 brief and Free Learning Resources & Support Guide.
- EDA Playground: https://www.edaplayground.com
- Icarus Verilog: http://iverilog.icarus.com
- GTKWave: http://gtkwave.sourceforge.net
- Verilog HDL language reference and sequential logic learning materials.

15. Submission Checklist

- Include RTL source files for all four modules.
- Include the testbench source file.
- Compile and simulate the design in EDA Playground or Icarus Verilog.
- Open the generated VCD file in GTKWave and capture the actual simulator/waveform screenshots if requested.
- Check that the observed outputs match the expected behavior described in this report.

14. Actual Simulation Evidence

The following screenshots were captured from the EDA Playground session using Icarus Verilog 12.0. The simulation log completed normally and generated the VCD waveform file task3.vcd.

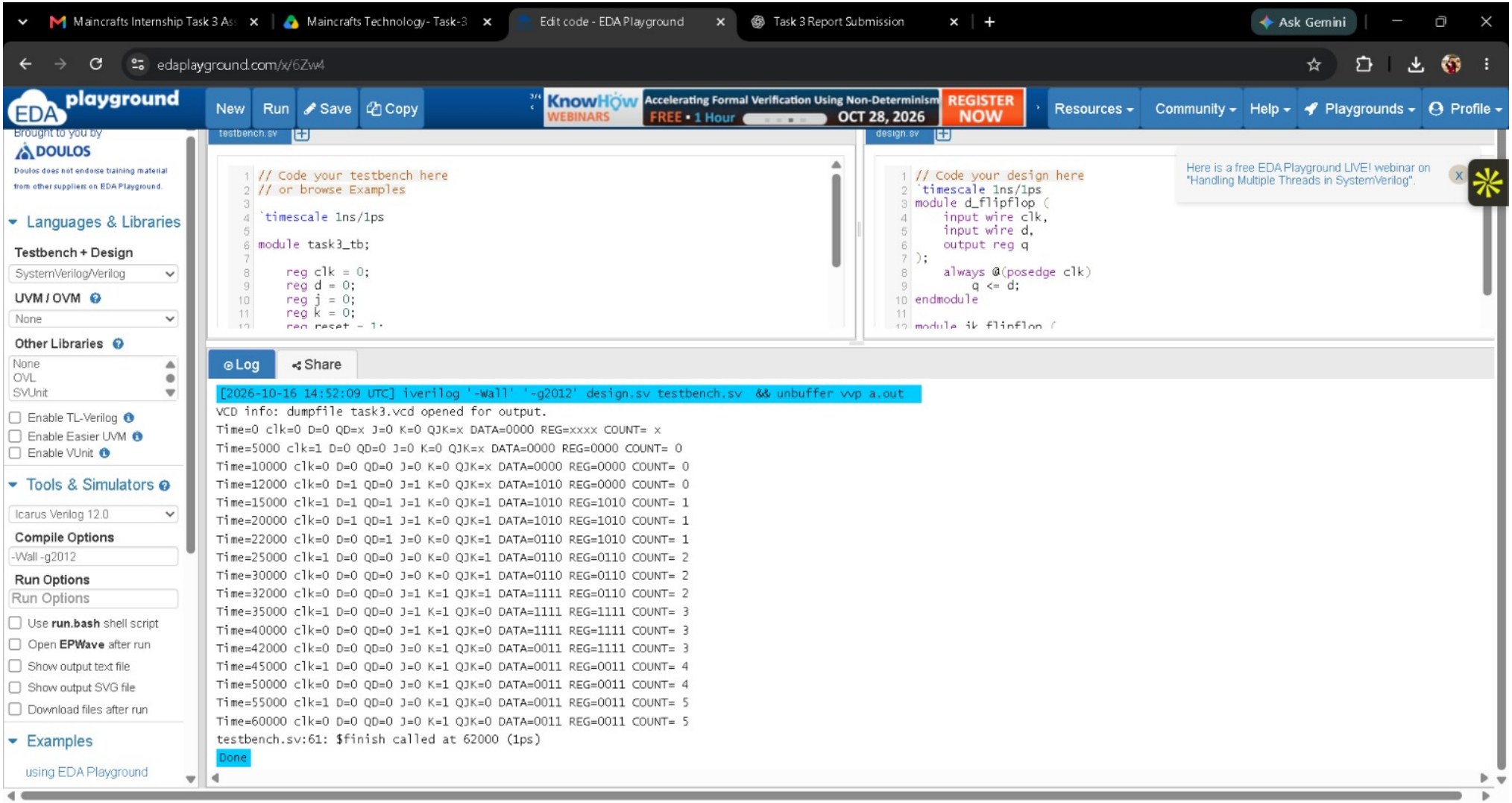


Figure 1. EDA Playground testbench and successful Icarus Verilog simulation log.

15. EPWave Timing Diagram

The waveform shows the clock, D and JK inputs and outputs, 4-bit input and register output, reset, and 4-bit counter across the 0–62,000 ps simulation interval. EPWave displays multi-bit buses in hexadecimal radix (A = 1010, 6 = 0110, F = 1111, and 3 = 0011).

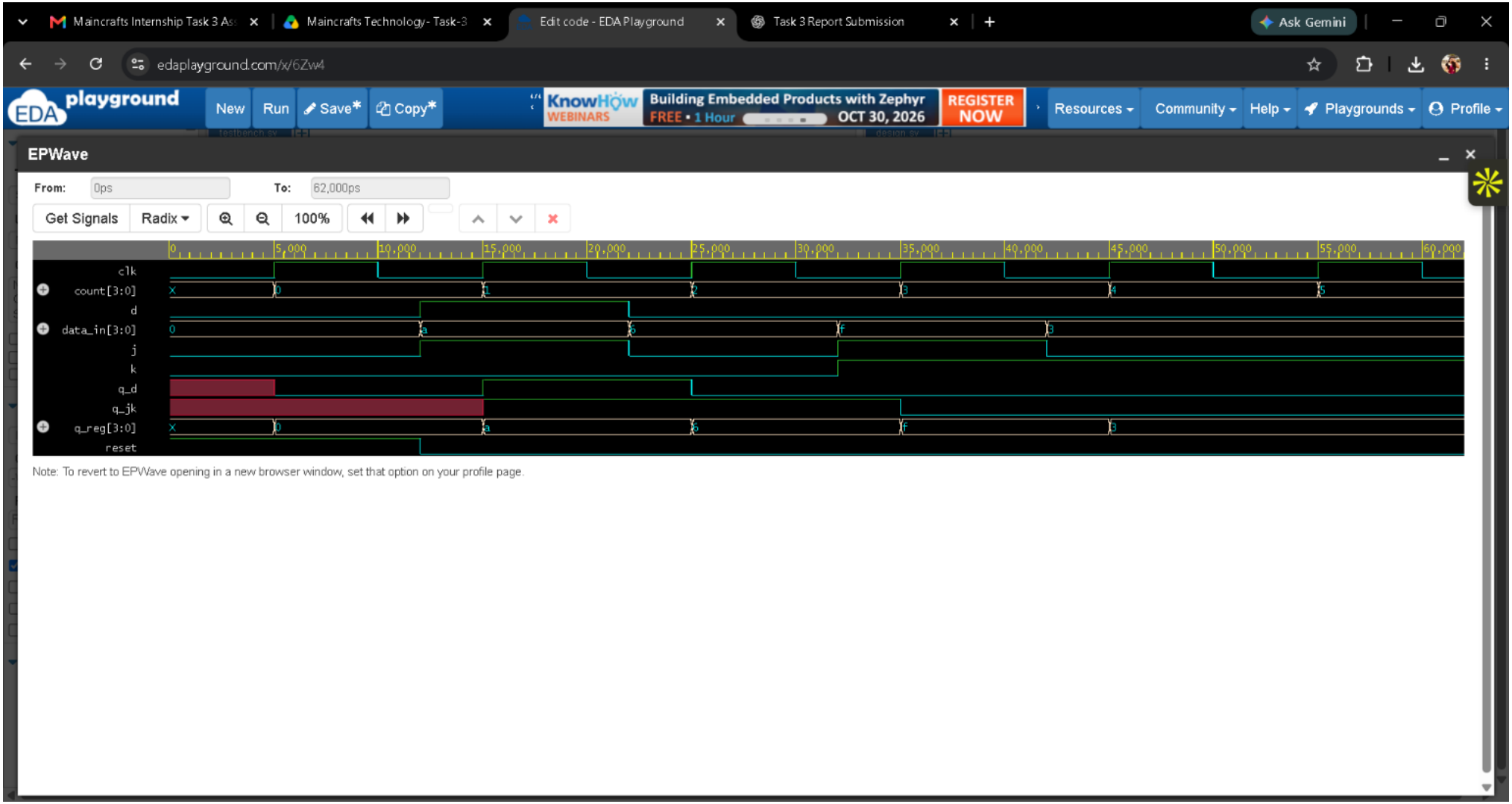


Figure 2. Actual EPWave waveform produced from task3.vcd.

16. Simulation Results and Discussion

D flip-flop: QD updates on rising clock edges according to D. **JK flip-flop:** the applied sequence demonstrates set (J=1, K=0), hold (J=0, K=0), toggle (J=1, K=1), and reset (J=0, K=1).

4-bit register: the register captures the applied bus values 1010, 0110, 1111, and 0011 on clock edges. In the waveform, these values appear as A, 6, F, and 3 because EPWave is using hexadecimal display.

4-bit counter: reset initializes the count to 0, after which the counter increments at rising clock edges through 1, 2, 3, 4, and 5 during the observed interval.

Completion: the log reports “$finish called at 62000 (1ps)” and “Done”, with no compilation errors shown. The earlier time-unit warning was no longer present in the submitted log screenshot.

Conclusion: The observed simulation log and waveform are consistent with the expected behavior of the four sequential circuit designs for the testbench stimulus used. The counter was observed through 5; this run does not demonstrate a full 16-state wraparound.